做了 dc cvau，最后却用 ic iallu——它只作废执行它的那个核的指令缓存。进程在一个核上装好，迁移到另一个核运行，那里的 I-cache 仍命中同一低地址上旧程序的指令：执行的是旧代码，用的却是新程序的数据和栈。

解决方案. 改为广播到 Inner Shareable 域的 ic ialluis，前面的逐行 dc cvau 和 dsb ish 保留（第 4 章故障二）。exec()、uvminit()、uvmcopy() 都经过这个函数。排查这类问题时，要用实际装进 SD 卡的那个 ELF 去反查 ELR，不能用另一次构建的符号文件。

问题 10：输出到 fixed WB/level 后长时间停顿

日志输出.

```
camshot: frame received 640x480, 384040 bytes
camshot: unpacking RAW10...
camshot: sensor test pattern; fixed WB/level
（长时间没有新输出）
```

问题分析. 看起来像转换死循环，但测试图案模式下转换并不做统计，下一步其实是以 O_TRUNC 打开已存在的 /boot/camera.bmp。截断要释放旧文件的簇链，旧的 fat_free_chain() 对每个簇都读一次 FAT、再对两份 FAT 各做一次读改写，覆盖一个 320×240 的 BMP 就要五百多次 SD 扇区事务，而且每次都是 PIO 轮询。

解决方案. fat_free_chain() 缓存当前 FAT 扇区，在其中批量清除属于这条链的项，换扇区时才把整扇区写到每份 FAT（6.8 节）。同时 camshot 在打开输出文件、写 BMP 头和开始转换之前各打印一行阶段日志，BMP 每 16 行写一次，并用版本串 converter fixed-pattern-v2 fat-progress batch16 确认 SD 卡上的程序已经更新——以后再停住，可以直接看出停在哪一步。

问题 11：新建文件停在 directory-entry begin

日志输出.

```
camshot: sensor test pattern; fixed WB/level
camshot: preparing output /boot/cavfs: boot create begin path=/boot/cam02.bmp sub=/cam02.bmp mode=601
m02.bmp (230454 bytes)...
vfs: boot create stat complete exists=0
vfs: boot create directory-entry begin
（此后再无输出）
```

问题分析. 先排除两个干扰：/boot/cavfs: ... m02.bmp 是用户态逐字符输出与内核 printf 交错的结果，路径本身没坏（7.9.10 小节）；exists=0 说明 stat 已经正常返回，阻塞点在 fat32createfile() 里。这和问题 10 不是同一个问题：exists=1 且停在 truncate begin 才是释放旧簇链慢。第一个猜测是“根目录要扩展，簇分配器扫描太慢”，于是先给 fat_alloc_cluster() 加了按扇区缓存和 FSInfo 提示，并加了 fat32: extending root directory 日志——真机上仍停在同一行，连新日志都没出现，说明根本没走到分配器。

真正的原因是内核栈。每个进程的内核栈只有 4 KiB，旧的 fat32createfile() 在栈上放了一个路径匹配结构、22 个目录槽和 256 字节的文件名，又嵌套调用 fat_find_path()，后者再放 256 字节文件名、520 字节 LFN 字符数组和 20 个目录槽。光这些数组就接近 3.5 KiB，加上 VFS、系统调用的栈帧和保存的寄存器，越过了 4 KiB。stat 只用到其中一层，所以没事。至于为什么是“停住”而不是一条报错：内核栈之间隔着不映射的保护页（第 4 章），越界先落进保护页、触发 EL1 数据中止；而 EL1 的异常入口要在同一个 SP_EL1 上压 272 字节保存现场（3.2 节），这一压又落在保护页里，异常入口自己再次中止，CPU 在异常向量里反复进出，串口上什么也不会出现。